

Nightreign Informational Website — Project Plan

Version: 1.2 **Date:** April 21, 2026 **Game:** Elden Ring Nightreign (v1.03.2 + DLC)

1. Project Overview

The goal is to transform a community-maintained spreadsheet of Elden Ring Nightreign game data into a fully interactive web application. The site will serve as a reference tool for players looking up weapon and relic passives, boss stats and weaknesses, character builds, consumable effects, and levelling costs.

The spreadsheet was created by Slay with contributions from Unlined-Betters, Emerald Wolf, and Penumbra, and covers version 1.03.2 including DLC content. Each tab will be downloaded manually as a CSV or XLSX file before running the data pipeline.

A core design goal beyond the initial build is **ongoing maintainability**: the site should be structured so that new patches, new characters, new bosses, or entirely new data categories can be incorporated with minimal friction — ideally just downloading updated files and re-running the pipeline, with no changes to the React frontend required for data additions.

2. Spreadsheet Inventory

The spreadsheet contains the following sections (tabs), each of which maps directly to a site section:

2.1 Talismans

Approximately 65 talismans with four columns: Name, In-Game Effect (display text), Actual Mechanical Effect (precise values), and Notes. Example: *Crimson Amber Medallion* increases HP by 1.12x.

2.2 Weapon / Equipment Passives

Passive effects granted by weapons and equipment. Columns: Category, In-Game Description, Actual Effect, Stackable With Self, Notes. Categories include Stat, Offensive, Defensive, and Utility.

2.3 Generic Relics

Relics that any character can equip. Same column structure as passives. Includes stat-boosting relics (Vigor +1/2/3, etc.) and combat relics (Physical Attack Up, improved critical hits, etc.).

2.4 Character-Specific Relics

Relics unique to each of the ten playable characters: Wylder, Guardian, Ironeye, Duchess, Raider, Revenant, Recluse, Executor, Scholar, and Undertaker. These alter each character's skills, art, or stats in unique ways.

2.5 Dormant Powers

Passive bonuses found as dormant powers on gear. Columns: Category, Power Name, In-Game Description, Actual Effect, Stackable, Notes. Includes exploration perks (rune bonuses), stat multipliers, and combat modifiers.

2.6 Boss Build Recommendations

A matrix of bosses (rows) x characters (columns). Each cell lists recommended relics and items for that boss/character combination. Covers all Nightlords and major field bosses.

2.7 Status Effects and Consumables

All in-game consumable items and status-inflicting tools. Columns: Name, In-Game Description, Actual Effect, Duration, Notes. Covers aromatics, boluses, grease, pots, and status ailments.

2.8 Character Base Stats

One row per character showing base-level stats: HP, FP, Stamina, Vigor, Mind, Endurance, Strength, Dexterity, Intelligence, Faith, Arcane, physical/elemental damage negations, and status resistances.

2.9 Character Level-Up Stats

Full stat breakdown for every character at every level (1-15), including all attributes, damage negations, HP, FP, and Stamina.

2.10 Boss HP Tables (Standard + DLC)

Six sub-tables covering boss HP at each difficulty tier: Standard Day 1, Day 2, Day 3, and DLC Day 1, Day 2, Day 3. Each row shows boss name, internal game ID, and health value. Also includes elemental resistance and immunity data per boss.

2.11 Rune / Level Cost Table

Rune costs to reach each level, cost increase per level, and cumulative total runes needed.

3. Proposed Site Architecture

Each spreadsheet tab maps to a page or sub-page. The site will use a persistent top navigation bar with the following structure:

Route	Page	Source Data
/	Home / Dashboard	Summary cards, current version badge, links
/talismans	Talismans	Tab 2.1
/relics	Relics & Passives	Tabs 2.2, 2.3, 2.4
/dormant-powers	Dormant Powers	Tab 2.5
/consumables	Consumables & Status Effects	Tab 2.7
/characters	Characters	Tabs 2.8, 2.9
/characters/:name	Character Detail	Tabs 2.4, 2.8, 2.9
/bosses	Bosses	Tabs 2.10
/bosses/:name	Boss Detail	Tabs 2.6, 2.10
/builds	Build Recommendations	Tab 2.6
/level-calculator	Level / Rune Calculator	Tab 2.11
/changelog	Patch Changelog	changelog.json (manually maintained)

3.1 Key Interactive Features

Global search bar — A single search input in the navigation bar that returns matches across all data types (talismans, relics, bosses, characters) with category labels.

Talismans page — Filterable table with a search box, category chips, and sortable columns. Each row expandable to show full effect text.

Relics & Passives page — Tabbed view: Generic Relics | Weapon Passives | Character Relics. Filter by character, category (Stat / Offensive / Defensive / Exploration), and stackability.

Characters page — Grid of character cards. Each character detail page shows: base stats table, per-level stat progression chart, and their unique relics.

Bosses page — Table with columns for boss name, HP at each day/tier, and elemental weaknesses colour-coded (weak = green, immune = red, neutral = grey). Click a boss to open the detail page.

Boss detail page — Shows full HP table, resistance values, and a character-specific build recommendation matrix pulled from Tab 2.6.

Build Recommendations page — Full boss x character matrix. Filter by character to see all bosses at once, or filter by boss to compare characters.

Level / Rune Calculator — Enter a current level and a target level; the tool calculates the total rune cost. Displays a running total table.

Version badge — The Home page and site footer display the current game version (e.g., “v1.03.2”) pulled from meta.json. When the pipeline is re-run after a patch, this updates automatically.

Changelog page — A simple page listing patch updates in reverse chronological order: version number, date, and a short description of what changed. This is maintained as a hand-edited changelog.json file.

4. Recommended Tech Stack

4.1 Summary

Layer	Technology	Rationale
Data pipeline	Python + pandas + openpyxl	Reads locally downloaded XLSX/CSV files, outputs clean JSON
Frontend framework	React (Vite)	Component model suits tab/table/filter UI
Styling	Tailwind CSS	Utility-first, fast dark-mode theming
Routing	React Router v6	Declarative, well-documented, standard
Tables	TanStack Table (React Table)	Sorting, filtering, pagination built-in
Charts	Recharts	Lightweight, React-native, good for stat curves
Deployment	GitHub Pages or Netlify	Free static hosting, no server needed

4.2 Why Static (No Backend Server)?

The game data is essentially immutable between patches. There is no need for a live database or API server. Instead:

1. Each spreadsheet tab is downloaded manually as a CSV or XLSX file and placed in the pipeline/data/raw/ folder.
2. A Python script reads those local files and writes structured JSON to the public/data/ folder.
3. React loads those JSON files from the public/ directory at runtime.
4. The entire site is a bundle of HTML, CSS, JS, and JSON that deploys to any static host.

This approach eliminates server costs, simplifies deployment, and makes the site extremely fast. When the spreadsheet is updated for a new game patch, download the updated files, re-run the Python pipeline, and redeploy — the whole process takes under a minute.

4.3 Why React over a Python Web Framework?

Flask and FastAPI are excellent choices when you need a live server — for dynamic data, user accounts, or database queries. For a reference site where all data is known at build time, React with Vite gives a better result: instant page transitions, client-side search and filtering with no network round-trips, and a faster development experience for highly interactive UI components like sortable tables and multi-filter panels. Python remains central to the project as the data pipeline layer.

4.4 Python Pipeline in Detail

The data pipeline is a standalone Python script (or small package) with no runtime dependencies on the web server. Its responsibilities are:

- Read locally downloaded XLSX or CSV files from `pipeline/data/raw/` via `pandas / openpyxl`
- Clean and normalise each tab into a typed Python dataclass
- Resolve cross-references (e.g., link a boss's build recommendation to the boss's resistance data)
- Export one JSON file per data type to `public/data/`
- Write `meta.json` with the current game version and date
- Validate the output schema with `Pydantic`

To update the site's data after a patch, download the updated spreadsheet tabs, place them in `pipeline/data/raw/`, and re-run: `python -m pipeline.main`

4.5 Designing for Updates

This section describes the specific technical decisions that make the site easy to maintain over time.

Data-driven React components. No boss names, character names, relic categories, or other game content should be hardcoded in the React source. Every list, table, filter option, and navigation item is derived from the JSON data. Adding a new boss to `bosses.json` should automatically make it appear on the Bosses page, the Build Recommendations matrix, and in global search — without touching any component code.

meta.json — version and last-updated info. The pipeline writes a small metadata file alongside the other JSON files containing the game version, last updated date, and whether DLC is included. The site reads this file to display the current version badge in the header and footer. Keeping it in JSON means it updates automatically whenever the pipeline is re-run.

Modular parsers. Each data type has its own parser file in `pipeline/parsers/`. Adding support for a brand-new spreadsheet tab (e.g., a new DLC adds a “Weapons” category that

did not exist before) means adding one new parser file and one new JSON output — the rest of the pipeline does not need to change.

Raw data archive. Old versions of the source files are moved to `pipeline/data/raw/archive/YYYY-MM-DD/` before each update rather than overwritten. This makes it straightforward to diff what changed between patches, which is useful both for writing changelog entries and for debugging parser regressions.

changelog.json — manually maintained patch notes. Rather than trying to auto-detect what changed between spreadsheet versions, the changelog is a hand-edited JSON file that you update alongside each patch. It contains one entry per version with the version number, date, and a list of notes describing what changed. This file lives in `public/data/changelog.json` and is the source for the `/changelog` page.

5. Data Models

The following JSON structures will be produced by the Python pipeline and consumed by the React frontend.

5.1 Talisman

Name, in-game effect description, actual mechanical effect, and notes. Example: Crimson Amber Medallion with effect “Increases HP by 1.12x”.

5.2 Relic / Passive

Category (Stat / Offensive / Defensive / Exploration), in-game description, actual effect, stackable-with-self flag, notes, and optional character field (null for generic relics, set to e.g. “Wylder” for character-specific ones).

5.3 Dormant Power

Category, power name, in-game description, actual effect, stackable-with-self flag, and notes.

5.4 Character

Name, base stats (HP, FP, Stamina, all attributes), base damage negations (physical, slash, strike, pierce, magic, fire, lightning, holy), base status resistances (poison, scarlet rot, hemorrhage, frostbite, sleep, madness, death blight), and an array of per-level stat objects covering levels 1-15.

5.5 Boss

Name, HP values at each tier (standard and DLC, days 1-3), elemental resistance values (numeric or “Immune”), and a per-character build recommendation object listing recommended relics and notes.

5.6 Consumable / Status Effect

Name, in-game effect description, actual mechanical effect, duration, and notes.

6. Implementation Phases

Phase 1 — Data Pipeline (Python)

Goal: Reliable, reproducible conversion of the locally downloaded spreadsheet files into JSON.

Tasks:

- Set up a Python project with virtual environment, pandas, openpyxl, and pydantic
- Download each spreadsheet tab manually as CSV or XLSX and place in pipeline/data/raw/
- Write a modular parser for each tab (one file per data type in pipeline/parsers/)
- Validate output with Pydantic schemas
- Export one JSON file per data type to public/data/, plus meta.json with the current version and date
- Create an initial public/data/changelog.json with the first entry for v1.03.2
- Write a README documenting the file naming conventions, the archive workflow, and how to re-run the pipeline

Deliverable: A pipeline/ folder containing the Python scripts, a pipeline/data/raw/ folder holding the source files, and a public/data/ folder with all generated JSON files including meta.json and changelog.json.

Phase 2 — React Project Setup

Goal: A running Vite + React + Tailwind project with routing and a placeholder for each page.

Tasks:

- Scaffold project with npm create vite@latest
- Install Tailwind CSS, React Router v6, TanStack Table, Recharts
- Set up a dark-themed layout with top navigation bar, mobile-responsive hamburger menu
- Copy JSON files from the pipeline into public/data/
- Create stub pages for all routes listed in Section 3

Deliverable: A running local dev server showing a skeleton of the site with all routes accessible.

Phase 3 — Core Pages

Goal: All pages fully populated with data, filterable, and searchable.

Tasks by page:

- **Talismans** — Render sortable/filterable table; implement name search, expandable rows
- **Relics & Passives** — Implement three-tab view (Generic / Weapon / Character); add category and character filters
- **Dormant Powers** — Category-filtered table
- **Consumables** — Searchable table with duration column and expandable notes
- **Characters** — Grid of character cards, detail page with stat table and level progression chart
- **Bosses** — Table with colour-coded resistance cells, HP by day/tier; detail page with build matrix
- **Build Recommendations** — Full matrix view with character and boss filters
- **Level Calculator** — Input current/target level, display rune cost with cumulative table
- **Changelog** — Render changelog.json as a reverse-chronological list of patch entries

Deliverable: All pages functional and populated with real data from the JSON files.

Phase 4 — Global Search & Polish

Goal: Cross-data search, visual polish, and quality-of-life features.

Tasks:

- Implement global search that queries across all JSON files and groups results by category
- Add keyboard shortcut (Cmd/Ctrl+K) to open search modal
- Tooltip on talisman and relic rows showing notes inline
- “Copy to clipboard” on build recommendations
- Mobile responsiveness pass — ensure tables scroll horizontally on small screens
- Favicon, page titles, meta descriptions
- Accessibility pass — keyboard navigation, ARIA labels on tables and filters

Deliverable: A polished, fully functional site ready for deployment.

Phase 5 — Deployment

Goal: Site live on a public URL.

Tasks:

- Choose hosting provider (see Section 7)
- Set up CI/CD with GitHub Actions: on push to main, build the React app and deploy
- Configure custom domain (optional)

- Test all pages on the live URL

Deliverable: Publicly accessible website.

7. Hosting Options

Option	Cost	Custom Domain	Auto-Deploy	Notes
GitHub Pages	Free	Yes (with CNAME)	Yes (GitHub Actions)	Best if code is already on GitHub
Netlify	Free tier	Yes	Yes (drag-drop or Git)	Easiest setup, great preview URLs
Vercel	Free tier	Yes	Yes (Git push)	Fastest CDN, good DX
Cloudflare Pages	Free	Yes	Yes	Best global performance

Recommendation: Start with Netlify. Connect your GitHub repository and Netlify will auto-deploy every time you push. The free tier has no practical limits for a site of this size. If you later want maximum performance, migrate to Cloudflare Pages with zero code changes.

8. Folder Structure

```

nightreign-site/ |— pipeline/ | |— data/ | |— raw/ (manually downloaded
CSVs/XLSX per tab) | |— archive/ (previous versions by date: YYYY-MM-DD/) | |—
parsers/ (one module per data type) | |— talismans.py | |— relics.py | |—
bosses.py | |— characters.py | |— consumables.py | |— models.py (Pydantic
data models) | |— export.py (writes JSON to public/data/, updates meta.json) | |—
main.py (entry point: python -m pipeline.main) | |— public/ | |— data/ (generated
JSON, committed to repo) | |— talismans.json | |— relics.json | |—
dormant_powers.json | |— characters.json | |— bosses.json | |— consumables.json
| |— meta.json (game version, last updated date) | |— changelog.json (hand-edited
patch notes) | |— src/ | |— components/ (SearchBar, DataTable, FilterChips,
CharacterCard) | |— pages/ | |— Home.jsx | |— Talismans.jsx | |—
Relics.jsx | |— DormantPowers.jsx | |— Consumables.jsx | |— Characters.jsx
| |— CharacterDetail.jsx | |— Bosses.jsx | |— BossDetail.jsx | |—
Builds.jsx | |— LevelCalculator.jsx | |— Changelog.jsx | |— hooks/ (useSearch,
useFilter, useData) | |— App.jsx | |— main.jsx | |— vite.config.js |—
tailwind.config.js |— package.json

```

9. Phase Checklist

Phase	Status
Phase 1 — Python Data Pipeline	☐
Phase 2 — React Project Setup	☐
Phase 3 — Core Pages	☐
Phase 4 — Global Search & Polish	☐
Phase 5 — Deployment	☐

10. Working with Claude Code

This project plan is designed to be handed off to Claude Code (the CLI tool) for implementation. The following notes will help Claude Code get up to speed quickly each session.

10.1 How to Start a Session

When opening Claude Code in the project directory, provide this document as context along with a brief instruction such as:

“Here is the project plan for the Nightreign website. We are currently on Phase X. The raw data files are in pipeline/data/raw/. Please continue from where we left off.”

10.2 Providing the Source Data

Before starting Phase 1, download each spreadsheet tab and place the files in pipeline/data/raw/. The recommended naming convention is to use the tab name as the filename, lowercased with underscores (e.g., talismans.csv, boss_hp.csv, character_stats.csv). Attach or reference these files when prompting Claude Code to write the parsers so it can inspect the actual column headers and data formats.

10.3 Suggested Phase Order

The best first step is the Python pipeline (Phase 1), because all subsequent work depends on clean, structured JSON. Once the JSON files exist, Phase 2 (React scaffold) takes under an hour, and pages can be built incrementally — starting with the simplest (Talismans) and working toward the most complex (Bosses with the build matrix and resistance colour-coding).

10.4 Patch Update Workflow

When a new game patch drops and the spreadsheet is updated, follow this sequence:

1. **Archive the current raw files.** Move the existing contents of pipeline/data/raw/ into pipeline/data/raw/archive/YYYY-MM-DD/ so you have a record of the previous version.

2. **Download the updated tabs.** Save the new CSV/XLSX files into `pipeline/data/raw/` using the same filenames as before.
3. **Re-run the pipeline.** Run `python -m pipeline.main`. This regenerates all JSON files and updates `meta.json` with the new version and date.
4. **Update the changelog.** Open `public/data/changelog.json` and add a new entry at the top describing what changed (new bosses, stat adjustments, new relics, etc.).
5. **Verify the output.** Spot-check a few JSON files or run the local dev server to confirm the new data appears correctly.
6. **Commit and deploy.** Commit the updated `public/data/` files and push — the site deploys automatically via CI.

If a patch introduces an entirely new data category (e.g., a new DLC tab that has never existed before), the additional step is to write a new parser in `pipeline/parsers/`, add a new JSON output file, and create the corresponding React page. The rest of the site continues to work unchanged.

Data sourced from the Nightreign Useful Info spreadsheet (v1.03.2 + DLC), created by Slay, Unlined-Betters, Emerald Wolf, and Penumbra.